4.7. Частотный анализ и поиск повторяющихся событий

4. Python в задачах информационной безопасности

В этом уроке

- Научимся проводить частотный анализ
- Познакомимся с модулем collections.Counter
- Применим анализ к текстам, логам, IP-адресам
- Будем выявлять подозрительные повторы

Частотный анализ

Частотный анализ — это метод, при котором подсчитывается, как часто повторяются элементы в данных.

Применяется к символам, словам, числам, IP-адресам, действиям и т.д.

Частотный анализ – это основа для обнаружения:

закономерностей,

повторяющихся шаблонов,

потенциальных угроз в системе.

```
text = "a a b c a b"
Counter(text.split())  # {'a': 3, 'b': 2, 'c': 1}
```

Метод может применяться к любым данным, от текстов до логов. Особенно полезен, когда мы не знаем, что искать, но подозреваем, что аномалии будут часто повторяться

Примеры применения в информационной безопасности

1. DDoS-атаки: один и тот же IP появляется сотни раз за короткое время.
2. Брутфорс: повторяющиеся попытки входа от одного пользователя.
3. Подозрительное поведение: частое использование одной и той же команды в логах терминала.
4. Анализ логов аудита безопасности — частое изменение одного и того же файла, пользователя и т.д.

Пример:

- IP 192.168.1.10 появляется в логах 500 раз за 10 минут — это аномалия.
- Пользователь admin делает 200 операций изменения настроек подряд.

Модуль collections.Counter

Counter — специальный класс из модуля collections.

Он позволяет легко подсчитывать элементы в последовательностях и возвращает словарь: {элемент: количество}.

Также класс работает с любыми итерируемыми объектами: списками, строками, файлами и т.п.

```
from collections import Counter
counts = Counter(['a', 'b', 'a', 'c'])
```

Counter — это обёртка над словарём, которая предоставляет удобные методы для анализа частоты. Особенно полезен для логов и текстов.

Простой пример использования Counter

```python
from collections import Counter

text = "a b a b c a"
counts = Counter(text.split())
print(counts)
```

```
Counter({'a': 3, 'b': 2, 'c': 1})
```

Что произошло?

- Разбили строку на список слов.
- Подсчитали каждое слово.

Counter работает с любым списком, а не только строками.

Метод most_common(), возвращает N наиболее частых элементов

Частотный анализ текста

Частотный анализ применим к текстам:

Письма, логи чатов, переписка, инструкции

Позволяет:

- Выделить ключевые слова
- Обнаружить спам/ботов
- Анализировать темы по частоте слов

Частотный анализ текста

Пример:

```
text = open("log_chat.txt").read()
words = text.lower().split()
word_counts = Counter(words)
print(word_counts.most_common(5))
```

```
[('error', 23), ('login', 17), ('failed', 15), ...]
```

Такие списки можно визуализировать в графиках или word cloud, или просто сохранять в отчёты.

Анализ логов средствами Counter

Логи — источник огромного объёма повторяющихся событий:

- Запросы к серверу
- Ошибки доступа
- Действия пользователей

```
with open("access.log") as f:
    lines = f.readlines()

counter = Counter(lines)
for line, count in counter.most_common(5):
    print(f"{count} раз: {line.strip()}")
```

Анализ повторяющихся IP-адресов в логах

В логах сервера часто фиксируются IP-адреса.

Повторяющиеся IP могут указывать на:

- автоматические сканеры,
- брутфорс-атаки,
- DDoS,
- подозрительную активность

```python
with open("access.log") as f:
    ips = [line.split()[0] for line in f]  # предполагаем, что IP — первый столбец
ip_counts = Counter(ips)
print(ip_counts.most_common(5))
```

```
[('192.168.0.5', 1523), ('10.0.0.1', 789), ...]
```

Часто встречающиеся IP могут быть блокированы, переданы в SIEM или помещены в список на дальнейшую проверку.

Визуализация частотных данных

Для визуализации можно использовать:

- Гистограммы (bar chart),
- Круговые диаграммы (pie chart),
- Словесные облака (word clouds).

Инструменты:

- matplotlib,
- seaborn,
- wordcloud.

```
import matplotlib.pyplot as plt

top_ips = ip_counts.most_common(5)
ips, counts = zip(*top_ips)

plt.bar(ips, counts)
plt.title("Top 5 IP addresses")
plt.xlabel("IP")
plt.ylabel("Requests")
plt.show()
```

Визуализация упрощает интерпретацию — особенно в отчётах.

Поиск подозрительных событий

Аномалии = события, встречающиеся слишком часто или слишком редко.

Примеры:

- один и тот же IP делает 10 000 запросов за 5 минут;
- один пользователь меняет 50 паролей подряд;
- файл passwd читается 100 раз за час.

Как выявлять?

```
suspicious = [ip for ip, count in ip_counts.items() if count > 1000]
```

- Использование threshold (порогов).
- Подсчёт повторений и сравнение со "средним".

Так можно сформировать списки для блокировки, отчёта или запуска автоматического реагирования.

Частотный анализ и выявление ботов

Боты часто используют:

- однотипные User-Agent’ы,
- одинаковые URL-запросы,
- повторяющиеся IP.

```
with open("web.log") as f:
    urls = [line.split()[6] for line in f]  # 7-й элемент — URL
url_counts = Counter(urls)
print(url_counts.most_common(5))
```

Повторяющийся URL с высокой частотой — признак брутфорса (перебора)

Частотный анализ действий пользователя

Также частотным анализом можно анализировать:

- команды в bash_history,
- действия в интерфейсе,
- изменения в базе данных.

```
commands = open("bash_history.txt").read().splitlines()
cmd_counts = Counter(commands)
print(cmd_counts.most_common(3))
```

Если одна команда используется подозрительно часто, возможно, это автоматический скрипт или вредоносная активность.

Ограничения метода частотного анализа

Частотный анализ эффективен, но не универсален.

Например, на него могут повлиять:

- Шум в логах
- Подделка IP-адресов
- Рандомизация со стороны злоумышленника
- Ложные срабатывания

То есть он не выявит DDoS или ложно сработает на дотошного пользователя, который делает одни и те же действия. Поэтому лучше его использовать в связке с корреляцией событий, временными интервалами и сигнатурным анализом.

Итоги урока

- Научились проводить частотный анализ
- Познакомились с модулем collections.Counter
- Применили анализ к текстам, логам, IP-адресам